Kafka

> "Kafka is a **message broker** — services send messages to it, other services read from it, asynchronously."

The Problem Without Kafka

In a monolith, services call each other directly:

```
credit-service ──HTTP──→ reporting-service
```

Problems:

- If reporting-service is down → credit-service fails too
- If credit-service sends 1000 events → reporting-service crashes
- Data lost if anything fails mid-call

The Solution With Kafka

```
credit-service ──→ [Kafka Topic] ──→ reporting-service
```

Benefits:

- credit-service doesn't care if reporting-service is down
- reporting-service processes at its own pace
- Messages are persisted — never lost
- Multiple consumers can read the same message

Kafka in one sentence

Apache Kafka is a **durable event log** that many services can read from **independently**.

Not a queue.
Not request/response.
A **log of facts**.

The only things you must remember

Event

An event = **something that happened**
Example:

```
CreditCreated
CreditApproved
PaymentReceived
```

Events are **immutable**.
If someone says “update event” → **incorrect**.

Topic

A topic = **a named list of events**

Think:

```
credit-events
payment-events
reporting-events
```

Kafka **keeps events** for hours / days / months.
They are **not deleted when read**.

> Topics store data in logs, and logs is sequence f data items we call them messages or events, so topics is where we store messages (they are immutable and if data change we make new topics reflect the transformed data so that why kafka does not have queues it has logs)

Producer

A producer = **who writes events**

Example:

```
credit-service → produces CreditCreated
```

That's it.
No response. No waiting.

Sends messages to a topic.

In your my project → `OutboxService`

```
kafkaTemplate.send(
    "credit.events",           // topic
    event.getAggregateId(),    // key
    event.getPayload()         // message
);
```

Consumer

A consumer = **who reads events**

Example:

```
reporting-service → reads CreditCreated

```

Consumers **pull** data when ready.

Offset (this is the trick)

Offset = "where I am in the list"

Kafka remembers:

```
reporting-service read until event #542

```

You can:

- replay
- restart
- rebuild data

This is why Kafka is powerful.

Consumer group

A group = **team of consumers**

Rule:

- One event → read by **only one consumer per group**
- Same topic → many groups allowed

So:

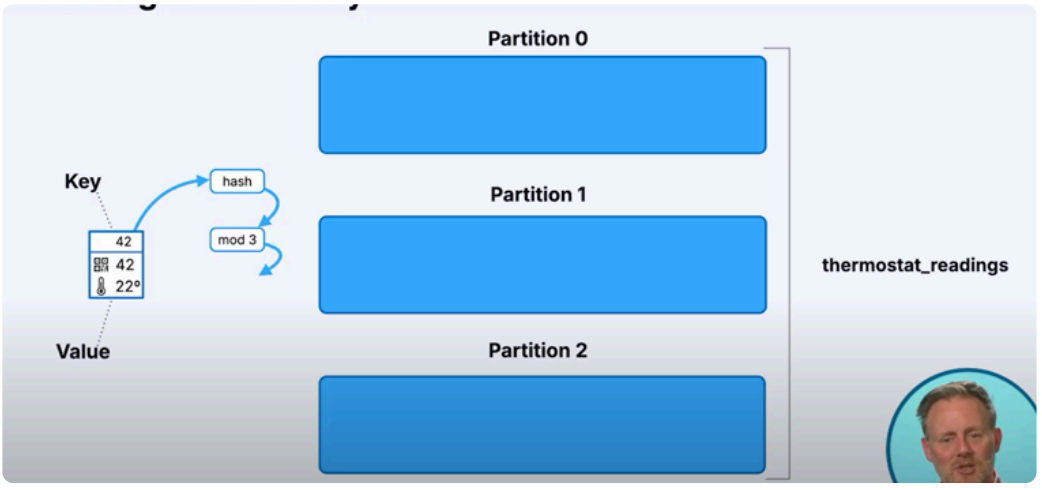
```
credit-events
 ├─ reporting-service (group A)
 ├─ audit-service     (group B)
 └─ alert-service     (group C)
```

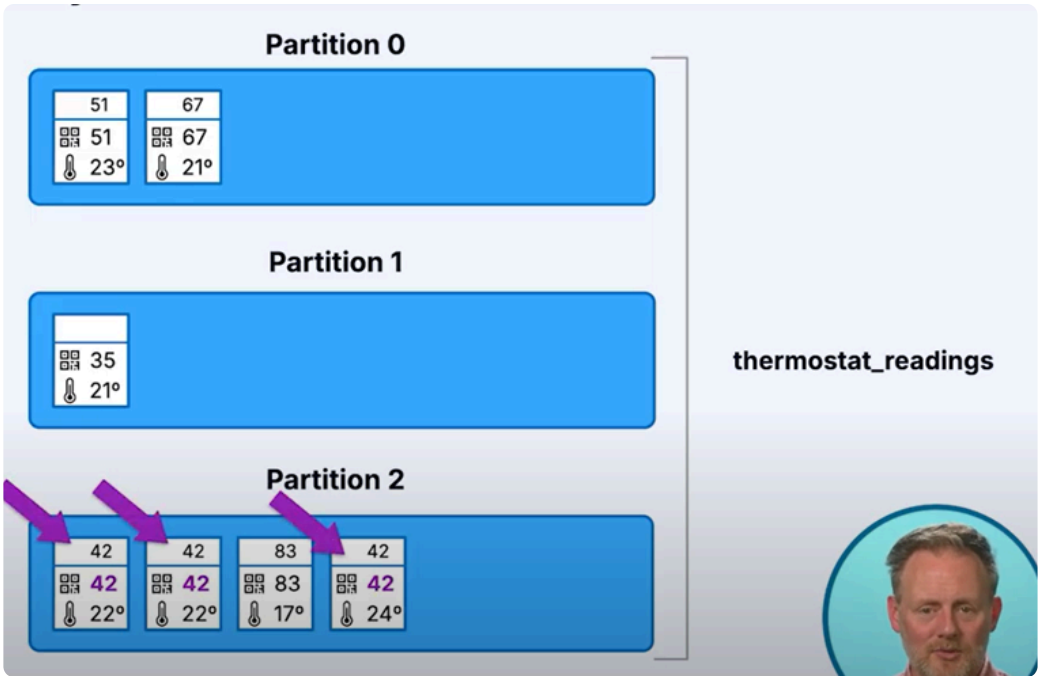
All get the same events.

No coupling.

Partitions

- The topic is partitioned, split into pieces
- Partitions is what make kafka scalable and reliable

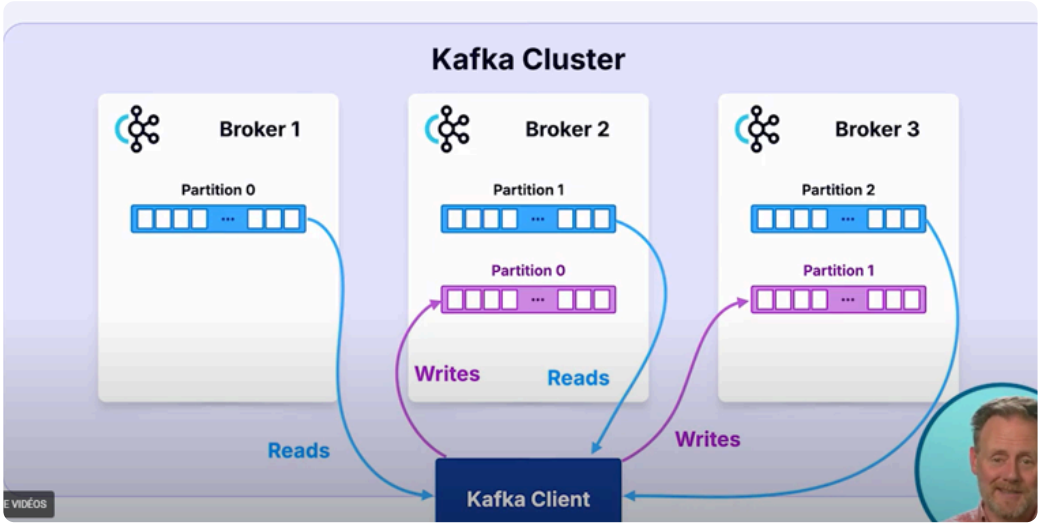




Partitions

Brokers

Kafka is composed of network of machines called brokers.they have access to local strorage.

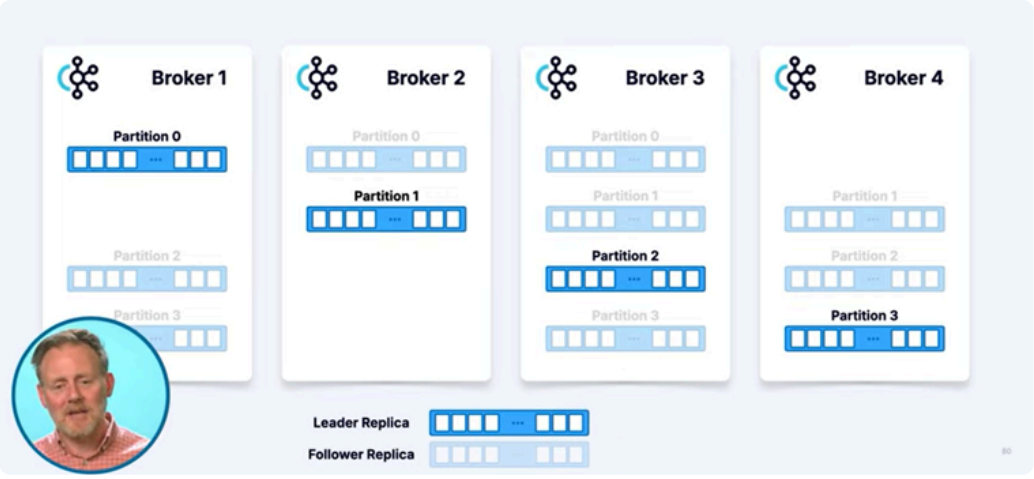


kafka cluster

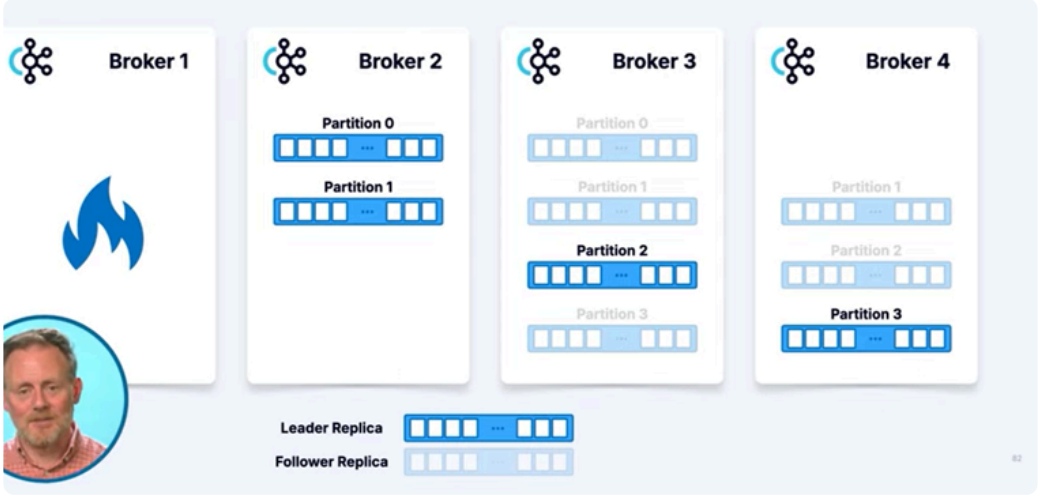
> kafka no longer has options to include zookeeper, brokers themselves do the work using Raft protocoles (kRaft)

Replication

- kafka replicate partitions across brokers:

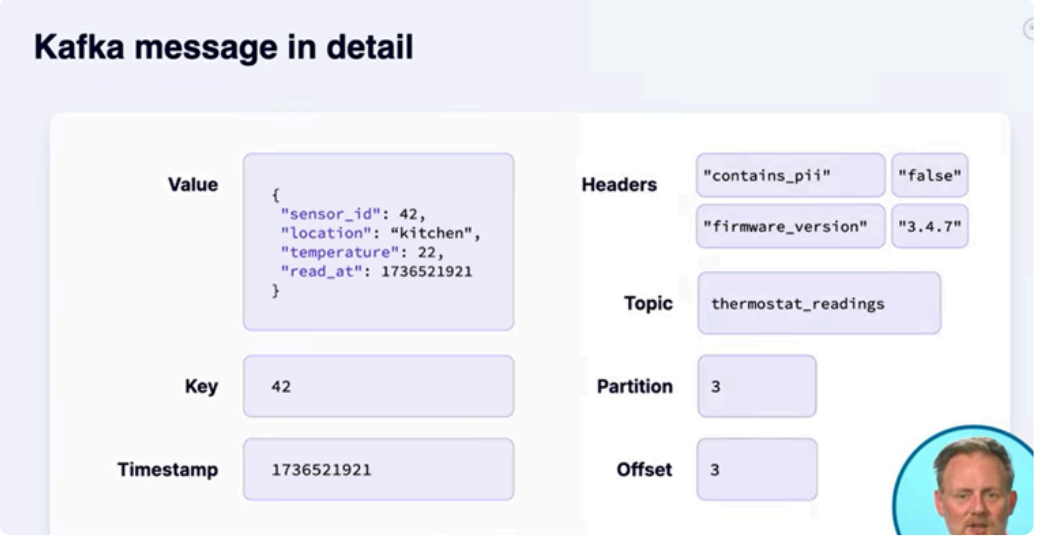


kafka replicate partitions across brokers



- Client write to and read from leaders but we can force to read or write from a follower if it is faster and y want performance

Kfka message:



Log retention and compaction

- Log **retention**: delete old data or trim logs by size
- Log **compaction:** keep only the latest value per key

One picture in your head (important)

```
credit-service
   |
   |  CreditCreated
   v
Kafka topic (log)
   |
   |---- reporting-service
   |---- audit-service
   |---- notification-service

```

Kafka sits **in the middle** as memory + buffer + history.

Kafka vs REST (ultra short)

REST	Kafka
Synchronous	Asynchronous
Tight coupling	Loose coupling
No history	Full history
One consumer	Many consumers

Term	Meaning
Topic	Named stream of events
Event	Immutable fact (`CreditApproved`)
Producer	Service that emits events
Consumer	Service that reads events
Offset	Position in the log
Partition	Parallelism unit
Consumer Group	Scaling mechanism

Outbox Pattern

Ce que c'est

Le problème à résoudre :

> Comment garantir qu'un event Kafka est publié **si et seulement si** la transaction DB métier est commitée ?

Sans outbox :

- DB OK ❌ Kafka KO → données perdues
- Kafka OK ❌ DB rollback → incohérence

Ce que TU as fait

Dans `credit-service` :

- Tu as une table `outbox_event`
- Dans la **même transaction** que `credit_lines`, tu fais :
  - `INSERT credit_lines`
  - `INSERT outbox_event (status = NEW)`
- Un **OutboxPublisher** (scheduler) :
  - lit `NEW`
  - publie vers Kafka
  - passe à `SENT`

Atomicité garantie.

Kafka Event

Ce que c'est

Un event = **fait métier passé**, immuable.

Ici :

```
CreditCreatedEvent
```

Ce que TU as fait

- Event **dans un module partagé (** `ledgerx-events` **)**
- Classe **immutable** (record)
- Pas de dépendance Spring / Kafka
- Payload JSON clair

Pourquoi c'est correct

- Découplage fort entre services
- Versionnable
- Testable
- Partage contrôlé

Consumer robuste

C'est quoi un consumer Kafka ?

Un consumer = un service qui **lit des messages Kafka** et fait quelque chose avec.

Problème :
Kafka **peut livrer le même message plusieurs fois** et **peut livrer un message mal formé**.

Objectif d'un consumer robuste

Un consumer robuste doit :

1. **Ne pas planter le service**
2. **Ne pas traiter deux fois le même message**
3. **Ne pas bloquer Kafka en boucle**
4. **Pouvoir redémarrer sans erreur**

Ce que TU as fait (simplement)

Tu consommes le message comme `String`

```
@KafkaListener(...)
public void onMessage(String payload)
```

Tu **contrôles toi-même** la désérialisation.

Pas de désérialisation automatique Kafka
Pas de crash Kafka

Tu désérialises manuellement

```
CreditCreatedEvent event =
    objectMapper.readValue(payload, CreditCreatedEvent.class);

```

Si c'est cassé → **TU sais pourquoi**.

Tu évites les doublons (idempotence)

```
if (processedEventRepository.existsById(eventId)) return;
```

Même message reçu 2 fois → traité 1 seule fois.

Tu commits l'offset seulement après succès

Kafka valide l'offset **après** la fin de la méthode.

Si exception → Kafka réessaie
Si succès → offset avancé

Un consumer robuste lit en String, valide manuellement, ignore les doublons, et ne bloque jamais Kafka.

Idempotence

Ce que c'est

Kafka **peut livrer plusieurs fois** le même message.

> "Au moins une fois" ≠ "une seule fois".

Ce que TU as fait

- Table `processed_events`
- Clé = `event_id`
- Check avant traitement
- Insert après succès

```
1 if (processedEventRepository.existsById(eventId)) return;
```

Pourquoi c'est correct

- Redémarrage safe
- Rebalance safe
- Retry safe

Read-model (Reporting DB)

Ce que c'est

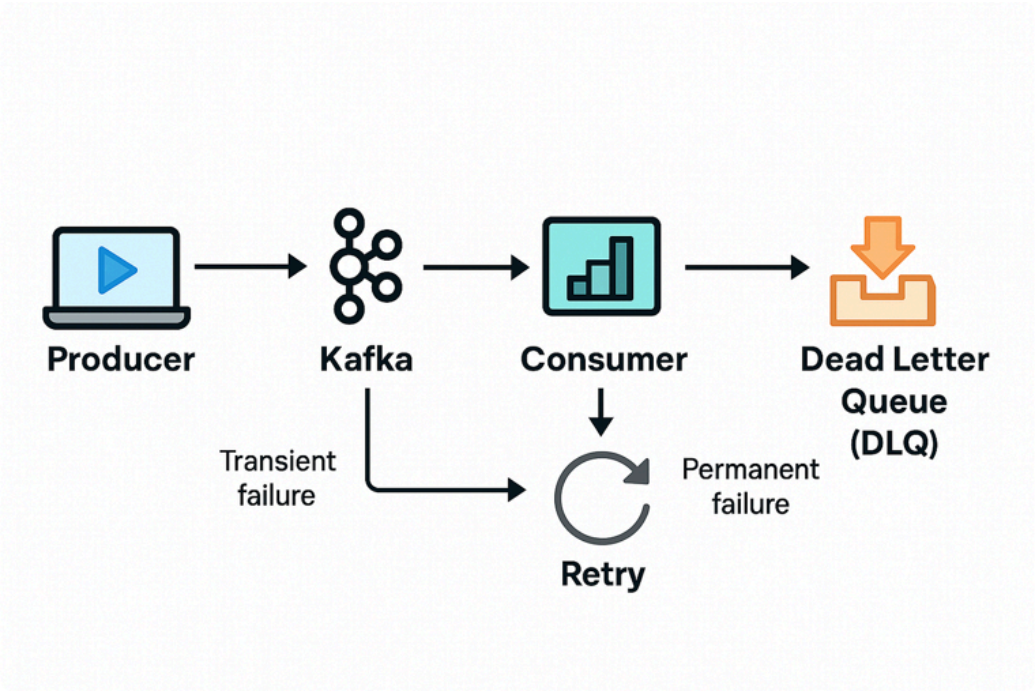
CQRS = **séparer écriture et lecture**.

Ce que TU as fait

- `credit_reports` **dans reporting-service**
- Alimentée **uniquement par Kafka**
- Pas d'accès DB cross-service

Pourquoi c'est correct

- Reporting rapide
- Zéro couplage
- Scalabilité
- Historisation possible



Définition rigoureuse

Aggregate (ex: Credit)

- Objet **vivant**
- Évolue dans le temps
- A **une seule identité**

```
Credit (aggregate)
  └── creditId = 123

```

Event (ex: CreditCreatedEvent)

- Fait **passé**
- **IMMUTABLE**
- Se produit **une fois**
- Peut être rejoué 1000 fois

```
CreditCreatedEvent
  ├── eventId      = e-001   (unique, pour toujours)
  ├── aggregateId  = 123     (creditId)
  ├── occurredAt
  
```

👉 **Un aggregate produit plusieurs events**

👉 **Un event appartient à UN aggregate**

```
✅ Vérité
ProcessedEvent est la mémoire du passé de ton consumer.
```

🧠 LedgerX – Kafka Architecture & Event Flow (Technical Summary)

🎯 Objectif global

Dans LedgerX, Kafka est utilisé pour :

- **découpler** les microservices
- **propager des événements métier** de manière asynchrone
- **garantir la cohérence** entre services **sans transactions distribuées**

Deux services clés :

- **credit-service** → *source de vérité / producer*
- **reporting-service** → *read-model / consumer*

1️⃣ Problème fondamental à résoudre

Kafka garantit **at-least-once delivery**.

Donc, sans précaution :

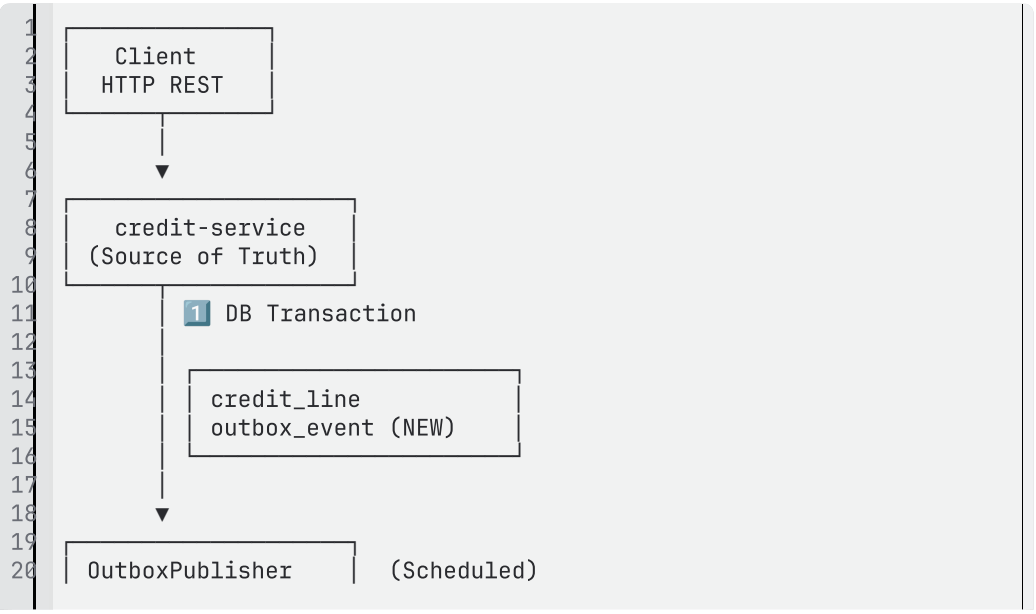
- un message peut être **rejoué**
- un service peut **crasher**
- un offset peut être **reset**

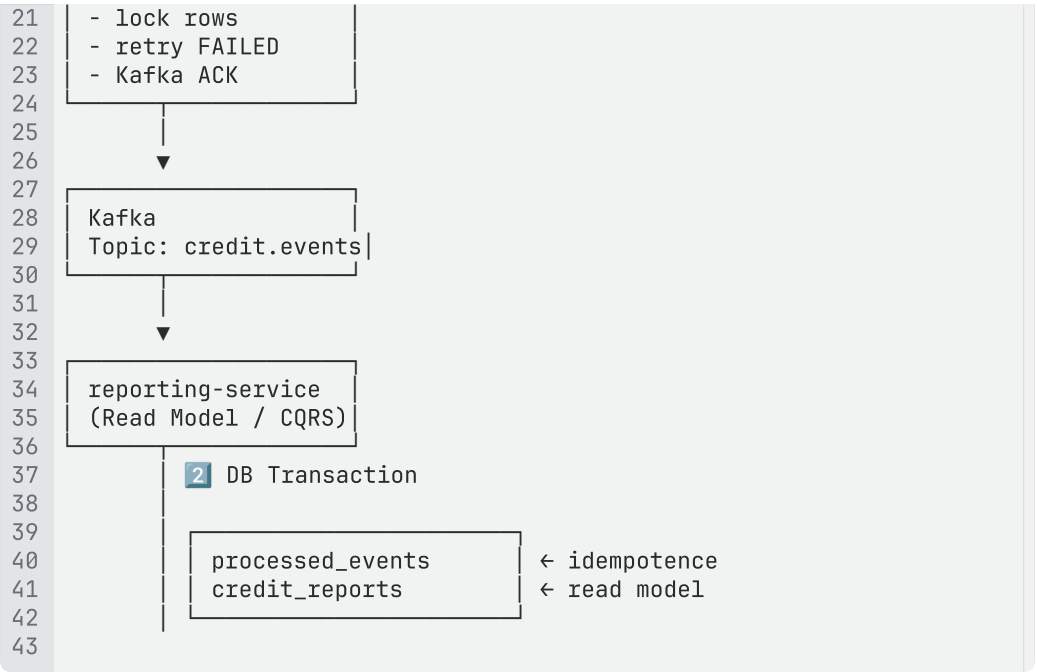
👉 **Les doublons sont normaux**
👉 **La cohérence doit être gérée par l'application**

LedgerX résout ça avec :

- **Outbox Pattern** côté producer
- **Idempotence DB-first** côté consumer

2️⃣ Schéma global LedgerX (flux réel)





3️⃣ credit-service – Kafka Producer (Outbox Pattern)

🎯 Rôle

- Gérer le **métier**
- Être la **source de vérité**
- Publier des événements **sans perte ni doublon**

> 🌈 “Dans LedgerX, Kafka est utilisé comme un bus d’événements asynchrones.
> Le credit-service publie des événements via un Outbox Pattern pour garantir la cohérence DB/Kafka.
> Le reporting-service consomme ces événements de manière idempotente grâce à une table de déduplication, ce qui permet de supporter les replays Kafka sans doublons.
> La base de données reste la source de vérité, Kafka sert uniquement au transport.”